

一、HW12-1

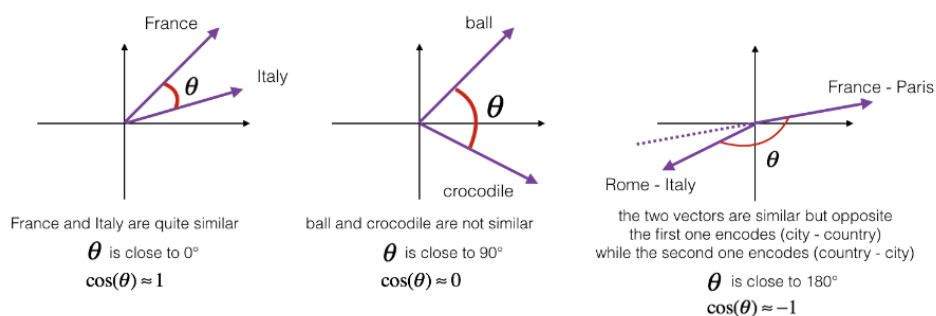
```
1 words, word_to_vec_map = read_glove_vecs('data/glove.6B.50d.txt')
```

觀察此 txt 檔案可以看到是一個已經寫好的 GloVe 檔案，字詞轉換成向量且非 one hot 型態，將 words、word_to_vec_map 個別 print 出。

1 words	1 word_to_vec_map
{'42.40', 'eltingville', '10a-10', '4,423', 'mithoff', 'substantiating', '24,200', 'niagara-on-the-lake', 'german-americans', 'pils', 'countersuing', 'mambalam', 'macaque', 'lonseny', 'willbros', 'thumbs-up', 'buzzed', '8.39', 'wallachians',	{'the': array([4.1800e-01, 2.4968e-01, -4.1242e-01, 1.2170e-01, 3.4527e-01, -4.4457e-02, -4.9688e-01, -1.7862e-01, -6.6023e-04, -6.5660e-01, 2.7843e-01, -1.4767e-01, -5.5677e-01, 1.4658e-01, -9.5095e-03, 1.1658e-02, 1.0204e-01, -1.2792e-01, -8.4430e-01, -1.2181e-01, -1.6801e-02, -3.3279e-01, -1.5520e-01, -2.3131e-01, -1.9181e-01, -1.8823e+00, -7.6746e-01, 9.9051e-02, -4.2125e-01, -1.9526e-01, 4.0071e+00, -1.8594e-01, -5.2287e-01, -3.1681e-01, 5.9213e-04, 7.4449e-03, 1.7778e-01, -1.5897e-01, 1.2041e-02, -5.4223e-02, -2.9871e-01, -1.5749e-01, -3.4758e-01, -4.5637e-02, -4.4251e-01, 1.8785e-01, 2.7849e-03, -1.8411e-01, -1.1514e-01, -7.8581e-01]), ,': array([0.013441, 0.23682, -0.16899, 0.40951, 0.63812, 0.47709, -0.42852, -0.55641, -0.364, -0.23938, 0.13001, -0.063734, -0.39575, -0.48162, 0.23291, 0.090201, -0.13324, 0.078639, -0.41634, -0.15428, 0.10068, 0.48891, 0.31226, -0.1252, -0.037512, -1.5179, 0.12612, -0.02442, -0.042961, -0.28351, 3.5416, -0.11956, -0.014533, -0.1499, 0.21864, -0.33412, -0.13872, 0.31806, 0.70358, 0.44858, -0.080262, 0.63003, 0.32111, -0.46765, 0.22786, 0.36034, -0.37818, -0.56657, 0.044691, 0.30392]),

可見 words 是一串無意義亂打的字詞，word_to_vec_map 則是 word embedding 完成的向量。

$$\text{CosineSimilarity}(u, v) = \frac{u \cdot v}{\|u\|_2 \|v\|_2} = \cos(\theta)$$



```
cosine_similarity(father, mother) = 0.8909038442893615
cosine_similarity(ball, crocodile) = 0.2743924626137942
cosine_similarity(france - paris, rome - italy) = -0.6751479308174201
```

我們可以藉由 cosinesimilarity 去計算兩者詞彙的相近程度，若 cos(theta)趨近於 0 表不相似(ball,crocodile)、趨近於 1 表相似(father,mother)，趨近於-1 表相反詞(France-paris,rome-italy)。

```

def complete_analogy(word_a, word_b, word_c, word_to_vec_map):
    """
    Performs the word analogy task as explained above: a is to b as c is to _____.

    Arguments:
    word_a -- a word, string
    word_b -- a word, string
    word_c -- a word, string
    word_to_vec_map -- dictionary that maps words to their corresponding vectors.

    Returns:
    best_word -- the word such that v_b - v_a is close to v_best_word - v_c, as measured by cosine similarity
    """

    # convert words to lower case
    word_a, word_b, word_c = word_a.lower(), word_b.lower(), word_c.lower()

    ### START CODE HERE ###
    # Get the word embeddings v_a, v_b and v_c (~1-3 lines)
    e_a, e_b, e_c = word_to_vec_map[word_a], word_to_vec_map[word_b], word_to_vec_map[word_c]
    ### END CODE HERE ###

    words = word_to_vec_map.keys()
    max_cosine_sim = -100 # Initialize max_cosine_sim to a large negative number
    best_word = None # Initialize best_word with None, it will help keep track of the word to output

    # Loop over the whole word vector set
    for w in words:
        # to avoid best_word being one of the input words, pass on them.
        if w in [word_a, word_b, word_c]:
            continue

        ### START CODE HERE ###
        # Compute cosine similarity between the vector (e_b - e_a) and the vector ((w's vector representation) - e_c) (~1
        cosine_sim = cosine_similarity(word_to_vec_map[w], e_c)

        # If the cosine_sim is more than the max_cosine_sim seen so far,
        # then: set the new max cosine sim to the current cosine_sim and the best_word to the current word (~3 lines)
        if cosine_sim > max_cosine_sim:
            max_cosine_sim = cosine_sim
            best_word = w
        ### END CODE HERE ###

    return best_word

```

此 function 會將丟進去的詞彙藉由 `word_to_vec_map[word]` 轉成向量形式的 `e_word`，再經由 `cosinesimilarity` 計算相似程度，並限制 `max_cosine_sim`。

```

italy -> italian :: spain -> portugal
india -> delhi :: japan -> japanese
man -> woman :: boy -> girl
small -> smaller :: large -> larger

```

成果可以視為 if Man is to Woman as King is to _____，讓機器從 `words` 的亂打詞彙中藉由 `cosinesimilarity` 自行拼湊出最適合的答案。

二、HW12-2

Part I.

```
1 X_train, Y_train = read_csv('data/train_emoji.csv')
2 X_test, Y_test = read_csv('data/tesss.csv')
3 maxLen = len(max(X_train, key=len).split())
```

1	I want to eat	4
2	he did not answer	3
3	he got a raise	2
4	she got me a present	0
5	ha ha ha it was so funny	2
6	he is a good friend	0

code	emoji	label
:heart:	❤️	0
:baseball:	⚾️	1
:smile:	😊	2
:disappointed:	😞	3
:fork_and_knife:	🍴	4

觀察 csv 檔案，裡面是評論句與 emoji label 的資料集，maxLen 是方便往後要轉換成 word embedding 的最大維度。

```
1 Y_oh_train = convert_to_one_hot(Y_train, C = 5)
2 Y_oh_test = convert_to_one_hot(Y_test, C = 5)
```

由於 label 只有 5 維，可以轉換成 one hot 形式，但在字詞方面還是得使用 word embedding。

```
1 word_to_index, index_to_word, word_to_vec_map = read_glove_vecs('data/glove.6B.50d.txt')
```

雖然是同一個 txt 檔案，但是 load 不一樣的數據，具有索引以及做好 word embedding 的字詞。

```
def sentence_to_avg(sentence, word_to_vec_map):
    """
    Converts a sentence (string) into a list of words (strings). Extracts the GloVe representation of each word
    and averages its value into a single vector encoding the meaning of the sentence.

    Arguments:
    sentence -- string, one training example from X
    word_to_vec_map -- dictionary mapping every word in a vocabulary into its 50-dimensional vector representation

    Returns:
    avg -- average vector encoding information about the sentence, numpy-array of shape (50,)
    """

    ### START CODE HERE ###
    # Step 1: Split sentence into list of lower case words (~ 1 line)
    words = [word.lower() for word in sentence.split(' ') if word != '']

    # Initialize the average word vector, should have the same shape as your word vectors.
    avg = np.zeros(word_to_vec_map[words[0]].shape)

    # Step 2: average the word vectors
    for w in words:
        avg += word_to_vec_map[w]
    avg = avg / float(len(words))

    ### END CODE HERE ###

    return avg
```

The diagram shows three word vectors: $e_I = \begin{pmatrix} 0.12 \\ 0.32 \end{pmatrix}$, $e_{love} = \begin{pmatrix} 0.75 \\ 0.11 \end{pmatrix}$, and $e_{you} = \begin{pmatrix} 0.54 \\ 0.13 \end{pmatrix}$. These are summed and then averaged to produce the final average vector: $avg = \begin{pmatrix} 0.47 \\ 0.19 \end{pmatrix}$.

我們輸入的 word 是一個句子所以得先切割成單字 sentence.split(' ')

For loop 幫助我們將每一個單字經由 word embedding 後的值做平均計算

```

1 def model(X, Y, word_to_vec_map, learning_rate = 0.01, num_iterations = 400):
2     """
3     Model to train word vector representations in numpy.
4
5     Arguments:
6     X -- input data, numpy array of sentences as strings, of shape (m, 1)
7     Y -- labels, numpy array of integers between 0 and 7, numpy-array of shape (m, 1)
8     word_to_vec_map -- dictionary mapping every word in a vocabulary into its
9     learning_rate -- learning_rate for the stochastic gradient descent algorithm
10    num_iterations -- number of iterations
11
12    Returns:
13    pred -- vector of predictions, numpy-array of shape (m, 1)
14    W -- weight matrix of the softmax layer, of shape (n_y, n_h)
15    b -- bias of the softmax layer, of shape (n_y,)
16    """
17
18    np.random.seed(1)
19
20    # Define number of training examples
21    m = Y.shape[0] # number of training examples
22    n_y = 5 # number of classes
23    n_h = 50 # dimensions of the GloVe vectors
24
25    # Initialize parameters using Xavier initialization
26    W = np.random.randn(n_y, n_h) / np.sqrt(n_h)
27    b = np.zeros((n_y,))
28
29    # Convert Y to Y_onehot with n_y classes
30    Y_oh = convert_to_one_hot(Y, C = n_y)
31
32    # Optimization loop
33    for t in range(num_iterations): # Loop over the number of iterations
34        for i in range(m): # Loop over the training examples
35
36            ### START CODE HERE ### (~ 4 lines of code)
37            # Average the word vectors of the words from the j'th training example
38            avg = sentence_to_avg(X[i], word_to_vec_map)
39
40            # Forward propagate the avg through the softmax layer
41            z = np.dot(W, avg) + b
42            a = softmax(z)
43
44            # Compute cost using the j'th training label's one hot representation
45            cost = -np.squeeze(np.sum(Y_oh[i] * np.log(a)))
46            ### END CODE HERE ###
47
48            # Compute gradients
49            dz = a - Y_oh[i]
50            dW = np.dot(dz.reshape(n_y, 1), avg.reshape(1, n_h))
51            db = dz
52
53            # Update parameters with Stochastic Gradient Descent
54            W = W - learning_rate * dW
55            b = b - learning_rate * db
56
57            if t % 100 == 0:
58                print("Epoch: " + str(t) + " --- cost = " + str(cost))
59                pred = predict(X, Y, W, b, word_to_vec_map)
60
61    return pred, W, b

```

$$z^{(i)} = W \times avg^{(i)} + b$$

$$a^{(i)} = \text{softmax}(z^{(i)})$$

$$\mathcal{L}^{(i)} = - \sum_{k=0}^{n_y-1} Y_{oh}_k^{(i)} * \log(a_k^{(i)})$$

Model function 輸入 X：句子、Y：label 需各自處理成單字並計算出 word embedding average 與 one hot，套入 loss 公式以及 forward propagation 但該 function 的 predict 是原作者已寫好的 function，而非 keras 的。

```

Epoch: 0 --- cost = 1.9520498812810072
Accuracy: 0.3484848484848485
Epoch: 100 --- cost = 0.07971818726014807
Accuracy: 0.9318181818181818
Epoch: 200 --- cost = 0.04456369243681402
Accuracy: 0.9545454545454546
Epoch: 300 --- cost = 0.03432267378786059
Accuracy: 0.9696969696969697
[[3.]
 [2.]
 [3.]
 [0.]
 [4.]]

```

predict 的結果是已將 one hot 轉成 label 值

```

1 X_my_sentences = np.array(["i adore you", "i love you", "funny lol", "lets play with a ball", "food is ready", "you are not
2 Y_my_labels = np.array([[0], [0], [2], [1], [4],[3]])
3
4 pred = predict(X_my_sentences, Y_my_labels , W, b, word_to_vec_map)
5 print_predictions(X_my_sentences, pred)

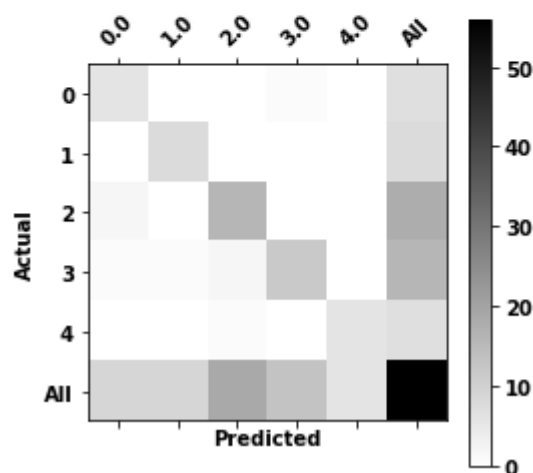
```

Accuracy: 0.8333333333333334

i adore you ❤️
i love you ❤️
funny lol 😄
lets play with a ball 🏈
food is ready 🍴
you are not happy 😞

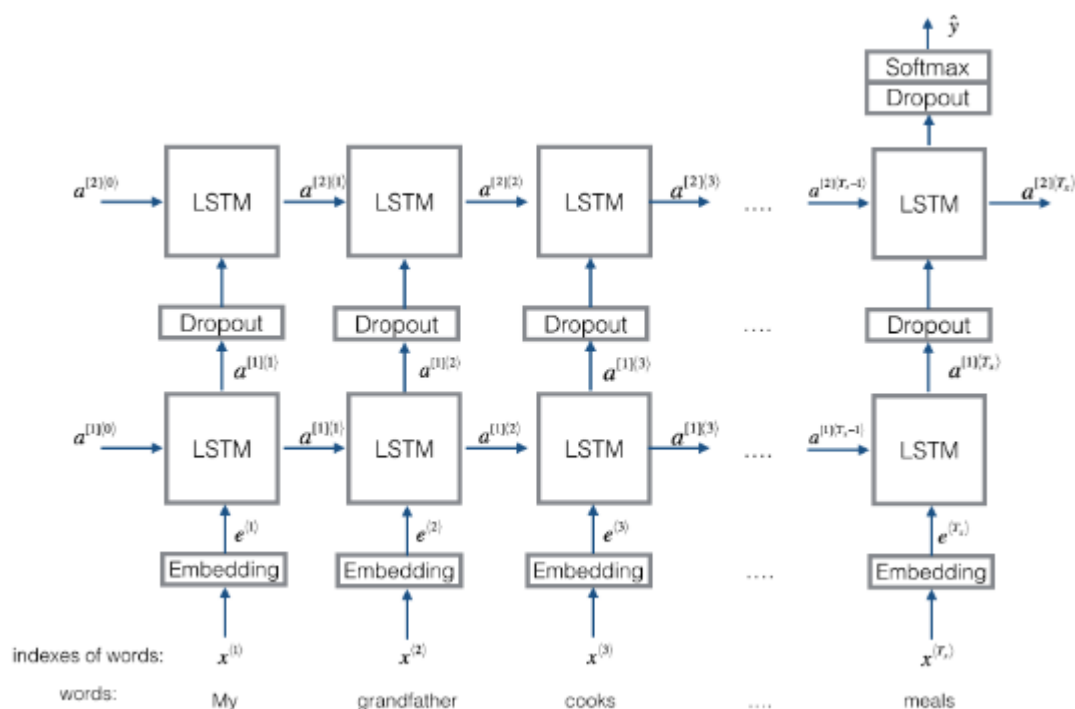
訓練好的 model 套用自己輸入的句子，可以看到尚不能辨識出否定句，僅能依靠字詞自身的意思去運算 label，無法辨識 not happy 是負面詞彙。

	❤️	🏈	😄	😞	🍴	
Predicted	0.0	1.0	2.0	3.0	4.0	All
Actual						
0	6	0	0	1	0	7
1	0	8	0	0	0	8
2	2	0	16	0	0	18
3	1	1	2	12	0	16
4	0	0	1	0	6	7
All	9	9	19	13	6	56

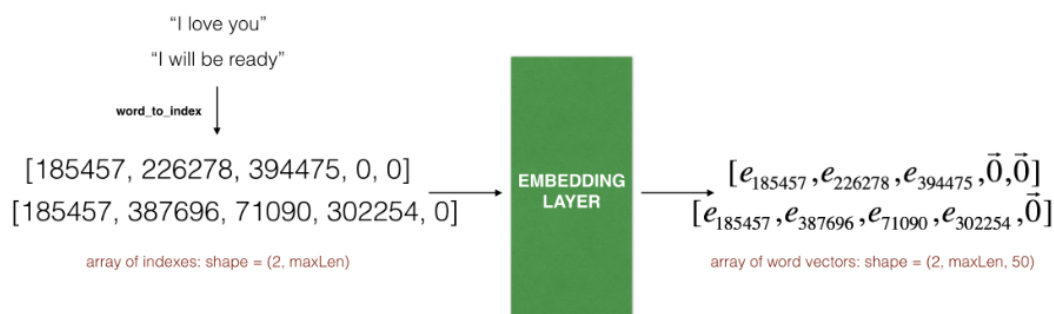


Confusion matrix(混淆矩陣)是作業第一次遇到，但是這裡的混淆矩陣並沒有網路上搜尋的困難難以理解，只是將預測與實際值做成表格。例如觀察 heart，hear label=0，predicted=actual 的有 6 個，屬於 TP(true predict)^[1]，而其他有 3 個是預測錯誤的。綜觀表格可以發現 heart 和 happy 錯最多，都有三個是錯誤的，以比例來說 heart 更高達 1/3，所以藉由觀察混淆矩陣，我們可以很清楚知道哪些表情的預測最需要改進，若表格數據龐大，還能藉由下方的有色圖快速判斷哪些地方的數值最高，分析是否為 TP 又或者是 FP。

Part II.



接下來我們需要使用 Keras 的 LSTM 來建構 Model，但 Keras 的 mini batch 需要有相同的維度，因此前面所取的 maxLen 就是用來處理這樣的情況，我們必須找到所有詞句最長的當作基準，其餘比他短的句子缺空部分補上 0 向量。



```
# Convert the training sentence in lower case and split it into words. You should get a list of words.
sentence_words = [word.lower().replace('\t', ' ') for word in X[i].split(" ") if word.replace('\t', ' ') != '']
```

輸入句子，拆成單字，對照索引，放入索引序號，維度=maxLen 多的位子補 0，進行 Embedding 轉成 word vector，此時一個單字就有 50 維，所以整體維度擴增為 $(m, \text{maxLen}, 50)$ ， m =句子數量。

特別需要注意的地方是後面跑 model 發現資料集中有些句不相干字詞($\backslash t$)，我們必須把它去除，因此使用 replace 替換成空格，當作切割處。

```

def pretrained_embedding_layer(word_to_vec_map, word_to_index):
    """
    Creates a Keras Embedding() layer and loads in pre-trained GloVe vectors.

    Arguments:
    word_to_vec_map -- dictionary mapping words to their GloVe vector representations
    word_to_index -- dictionary mapping from words to their indices

    Returns:
    embedding_layer -- pretrained layer Keras instance
    """

    vocab_len = len(word_to_index) + 1                # adding 1 to fit with Embedding() arguments
    emb_dim = word_to_vec_map["cucumber"].shape[0]    # define dimensionality of embedding vector

    ### START CODE HERE ###
    # Initialize the embedding matrix as a numpy array of zeros of shape (vocab_len, emb_dim)
    emb_matrix = np.zeros((vocab_len, emb_dim))

    # Set each row "index" of the embedding matrix to be the word embedding corresponding to the index.
    for word, index in word_to_index.items():
        emb_matrix[index, :] = word_to_vec_map[word]

    # Define Keras embedding layer with the correct output/input shape
    # Use Embedding(...). Make sure to set trainable=False.
    embedding_layer = Embedding(vocab_len, emb_dim, trainable=False)
    ### END CODE HERE ###

    # Build the embedding layer, it is required before setting the weights
    embedding_layer.build((None,))

    # Set the weights of the embedding layer to the embedding matrix
    embedding_layer.set_weights([emb_matrix])

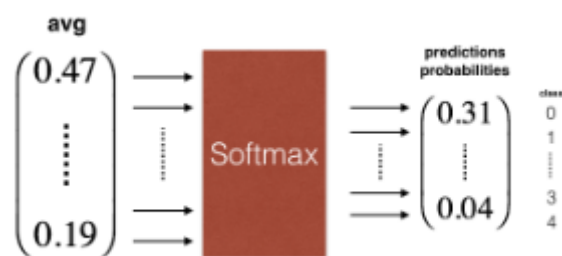
    return embedding_layer

```

這是一個 pretrained embedding layer，所以由 word_to_vec_map 這個已寫好的 word embedding 資料庫得到的維度=50，我們的 embedding matrix 需要的維度是(字彙長度,50)，for loop 將 word 經 word_to_vec_map 轉成 word embedding 再放入 embedding matrix，對應其 index，放入指定位置，製作好的 embedding matrix 是用來確保其權重在我們自己架設的 keras embedding layer 相同，這樣訓練出來的結果會與原作者 pretrained model 相同^[2]。

Model: "model_1"

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	(None, 10)	0
embedding_3 (Embedding)	(None, 10, 50)	20000050
lstm_1 (LSTM)	(None, 10, 128)	91648
dropout_1 (Dropout)	(None, 10, 128)	0
lstm_2 (LSTM)	(None, 128)	131584
dropout_2 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 5)	645
activation_1 (Activation)	(None, 5)	0
Total params: 20,223,927		
Trainable params: 223,877		
Non-trainable params: 20,000,050		



模型的架構：輸入訓練集需先經過 word embedding 轉成 word vector 才能讓機器看懂字句，變成能看懂的向量後進行訓練，這邊用的 layer 是 LSTM，加強版的 RNN 網路，其中 Dropout 為防止 overfitting，最後的 softmax 為預測 emoji 用。

```
132/132 [=====] - 0s 459us/step - loss: 0.0760 - accuracy: 0.9924
Epoch 48/50
132/132 [=====] - 0s 459us/step - loss: 0.0760 - accuracy: 0.9924
Epoch 49/50
132/132 [=====] - 0s 466us/step - loss: 0.0756 - accuracy: 0.9848
Epoch 50/50
132/132 [=====] - 0s 466us/step - loss: 0.0739 - accuracy: 0.9848
```

不同於 CNN 需要用大量平行運算，因為數據集很小所以 LSTM 跑的速度非常快，雖然 embedding 的結果與原作者相同，但在經過 LSTM layer 後權重並不能保證與原作者相同，所以模型還是可能出現 not happy 的 label 還是 heart 的情況發生


```
1 # Change the sentence below to see your prediction. Make sure all the words are in the Glove embeddings.
2 x_test = np.array(['you are not happy'])
3 X_test_indices = sentences_to_indices(x_test, word_to_index, maxlen)
4 print(x_test[0] + ' ' + label_to_emoji(np.argmax(model.predict(X_test_indices))))
5
```

you are not happy ❤️

Reference

1. <https://www.ycc.idv.tw/confusion-matrix.html> YC Chen 2017.08.04
2. <http://frankchen.xyz/2017/12/18/How-to-Use-Word-Embedding-Layers-for-Deep-Learning-with-Keras/>